

INFOB2DA | Practical Assignment 3 | Si-8

Irvan Ilahibaks 8081847 - Timon Baris 2173182 - Waling van Dongen 1176226

TA: Sita Newer Date: 13-10-2024

Task 1: Get dataset on screen

1.1 Explore the dataset similarly to the first assignment, so you have a good understanding of its content and features. Come up with an interesting one-minute story about the dataset.

```
1 #1.1 Get data on the screen
2 import pandas as pd
3
4 btData = pd.read_csv('blood_transfusion.csv') #Initialize data
```

```
1 btData.info() #viewing Data Types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 748 entries, 0 to 747
Data columns (total 5 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   months_since_last_donation            748 non-null    float64
1   total_number_of_donations             748 non-null    float64
2   total_blood_donated                   748 non-null    float64
3   months_since_first_donation           748 non-null    float64
4   class                                 748 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 29.3 KB
```

```
1 btData.describe() #summary statistics
```

	months_since_last_donation	total_number_of_donations	total_blood_donated	months_since_first_donation	class
count	748.000000	748.000000	748.000000	748.000000	748.000000
mean	9.506684	5.514706	1378.676471	34.282086	0.237968
std	8.095396	5.839307	1459.826781	24.376714	0.426124
min	0.000000	1.000000	250.000000	2.000000	0.000000
25%	2.750000	2.000000	500.000000	16.000000	0.000000
50%	7.000000	4.000000	1000.000000	28.000000	0.000000
75%	14.000000	7.000000	1750.000000	50.000000	0.000000
max	74.000000	50.000000	12500.000000	98.000000	1.000000

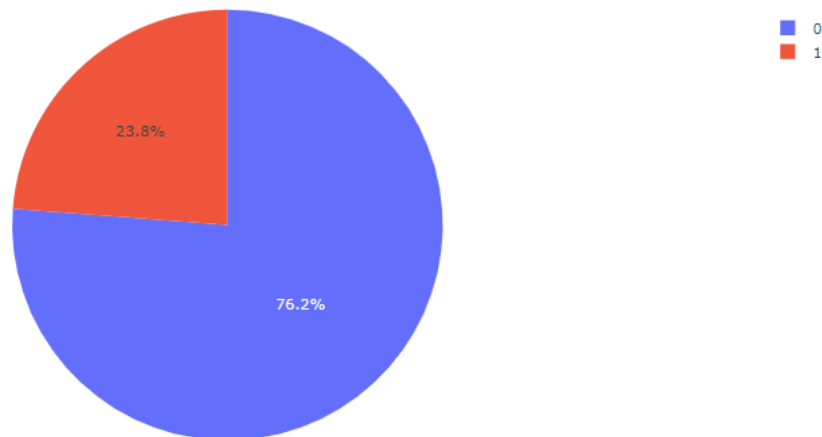
```
In [4]: 1 print(btData.to_string())
```

	months_since_last_donation	total_number_of_donations	total_blood_donated	months_since_first_donation	class
0	2.0	50.0	12500.0	98.0	1
1	0.0	13.0	3250.0	28.0	1
2	1.0	16.0	4000.0	35.0	1
3	2.0	20.0	5000.0	45.0	1
4	1.0	24.0	6000.0	77.0	0
5	4.0	4.0	1000.0	4.0	0
6	2.0	7.0	1750.0	14.0	1
7	1.0	12.0	3000.0	35.0	0
8	2.0	9.0	2250.0	22.0	1
9	5.0	46.0	11500.0	98.0	1
10	4.0	23.0	5750.0	58.0	0
11	0.0	3.0	750.0	4.0	0
12	2.0	10.0	2500.0	28.0	1
13	1.0	13.0	3250.0	47.0	0
14	2.0	6.0	1500.0	15.0	1
15	2.0	5.0	1250.0	11.0	1
16	2.0	14.0	3500.0	48.0	1
17	2.0	15.0	3750.0	49.0	1

The main focus of this assignment is to find out whether people will come back to donate blood again next month based on different gathered data. We started by looking at the information from all 748 people that have donated blood. We can see a strong majority

```
1 import plotly.express as px
2
3 #counting how many people there are of each class
4 peopleCount = btData['class'].value_counts().reset_index()
5
6 #naming the columns
7 peopleCount.columns = ['class', 'count']
8
9 # Create the pie chart using the counts
10 figpie = px.pie(peopleCount, values='count', names='class', title='People who came back (1) vs who did not (0)')
11 figpie.show()
12
```

People who came back (1) vs who did not (0)



belonging to class 0. This means over 570 people have not come back to donate, while only 178 people did. It is important to know that the people who are part of class 0 have not come back to donate yet. This does not mean that they will never come back, but that at the time of recording this data they have not come back yet. These people may come back to donate in the future. The people who came back to donate have 1388 donations in total, meanwhile people who have not come back yet have donated 2737 times. When looking at other data we see that 684.25 thousand mL of donated blood belongs to the 570 people who did not come back. The 178 people that did come back to donate were responsible for 347 thousand mL of blood. After this we took the average amount of donated blood per person in each class, seeing that the people who came back did on average almost 2 thousand mL per person while the people who did not come back did just over 1 thousand mL per person.

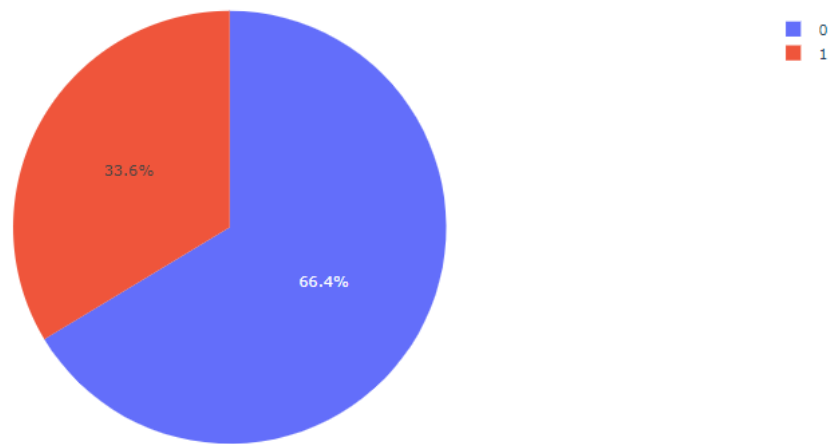
1.2 Create a visualization, which shows the characteristics of tuples of class 1, compared to tuples of class 0.

1.2.1 The amount of people who came back for blood donations and who didn't.

1.2.2 Total amount of donations per class.

```
1 # Grouping the data by class and summing the total blood donations
2 totalBloodDon = btData.groupby('class')['total_number_of_donations'].sum().reset_index()
3 totalBloodDon.columns = ['class', 'count']
4
5 figpie = px.pie(totalBloodDon, values='count', names='class', title='Amount of blood donations by each class')
6 figpie.show()
7
```

Amount of blood donations by each class



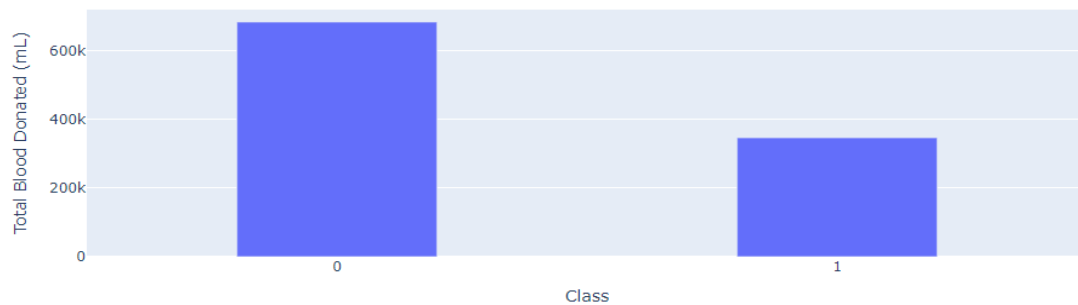
1.2.3 Total blood donated per class.

```

1 # Grouping the data by class and summing the total blood donated
2 totalBlood = btData.groupby('class')['total_blood_donated'].sum().reset_index()
3
4 # Create the bar chart
5 fig = px.bar(totalBlood, x='class',
6             y='total_blood_donated',
7             title="Total Blood Donated by Each Group",
8             labels={'class': 'Class', 'total_blood_donated': 'Total Blood Donated (mL)'},
9             )
10
11 fig.update_layout(
12     xaxis=dict(tickvals=[0, 1], range=[-0.5, 1.5]),
13     height=400,
14     bargap=0.6)
15
16 fig.show()
17

```

Total Blood Donated by Each Group



1.2.4 The average amount of donated blood per person per class:

```

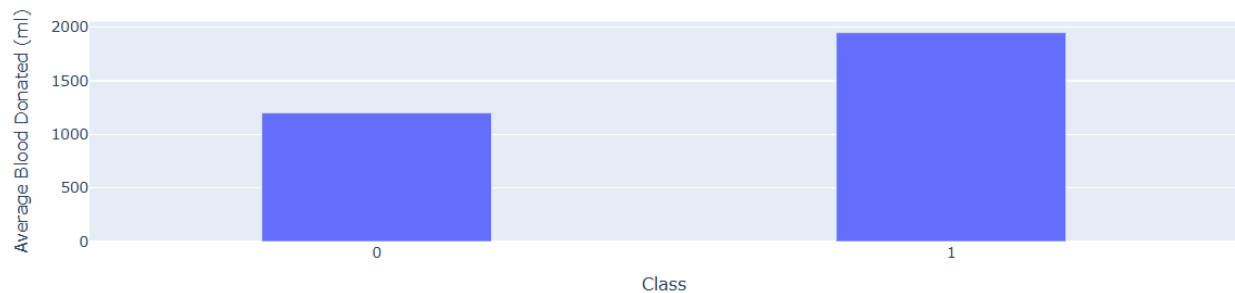
averageBlood = btData.groupby('class')['total_blood_donated'].mean().reset_index()

fig = px.bar(averageBlood, x='class', y='total_blood_donated', title="Average Blood Donated by Each Class",
             labels={'class': 'Class', 'total_blood_donated': 'Average Blood Donated (mL)'})

fig.update_layout(
    xaxis=dict(tickvals=[0, 1], range=[-0.5, 1.5]),
    bargap=0.6)
)

```

Average Blood Donated by Each Class



Task 2: Preprocessing

The first thing to check is whether there is any missing data. After checking for empty data cells, the data is standardized. If you're using distance-based algorithms (such as SVM, KNN, or Neural Networks), standardization is recommended because it handles outliers better and helps the algorithm by centering the data. For tree-based algorithms (like Decision Trees or Random Forests), both approaches can be used, but normalization is often preferred. Tree-based methods are generally not affected by the scale of the features.

```
1 from sklearn.preprocessing import StandardScaler
2
3 features = btData.drop('class', axis=1)
4
5 standardSC = StandardScaler()
6
7 standardFeatures = standardSC.fit_transform(features)
8
9 btSDData = pd.DataFrame(standardFeatures, columns=features.columns)
10
11 print("StandardScaler Features:")
12 print(btSDData.head())
```

```
StandardScaler Features:
  months_since_last_donation  total_number_of_donations  total_blood_donated \
0                -0.927899                7.623346                7.623346
1                -1.175118                1.282738                1.282738
2                -1.051508                1.796842                1.796842
3                -0.927899                2.482313                2.482313
4                -1.051508                3.167784                3.167784

  months_since_first_donation
0                2.615633
1                -0.257881
2                0.029471
3                0.439973
4                1.753579
```

```
1 #Task 2: Preprocessing
2
3 btData.isnull().sum() #The data below shows that every cell of data has data

months_since_last_donation    0
total_number_of_donations    0
total_blood_donated          0
months_since_first_donation  0
class                        0
dtype: int64
```

Task 3: Creating a train and test set

What defines a good train-test split?

- The training set should have enough data for the model to learn and recognize the patterns better, that is why data splits usually have around 70-80% of training data. The testing data should also be enough to properly show the accuracy of the model.
- Data from the testing data and training data should be clearly divided.
- The data should also be randomly split to assure there is no existing bias.

3.1 Look for a function in the Sklearn documentation which can create train-test splits. Use this function to create two train-test splits of different sizes.

```
1 #3.1: Creating a train and test set
2
3 from sklearn.model_selection import train_test_split
4
5 # Separate features and the target class
6 X = btsData
7 y = btData['class'] # Target
8
9 # Create 80-20 train-test split
10 X_train_80, X_test_80, y_train_80, y_test_80 = train_test_split(X, y, test_size=0.2, random_state=
11
12 # Create 70-30 train-test split
13 X_train_70, X_test_70, y_train_70, y_test_70 = train_test_split(X, y, test_size=0.3, random_state=
14
15 # Compare the results of the trained set to the test set and round the result to 3 decimaes to ma
16 print(f"80/20 train test split\nTrain set {y_train_80.value_counts(normalize=True).round(3).to_dic
17 print(f"70/30 train test split\nTrain set {y_train_70.value_counts(normalize=True).round(3).to_dic
18
```



```
80/20 train test split
Train set {0: 0.764, 1: 0.236}
Test set {0: 0.753, 1: 0.247}

70/30 train test split
Train set {0: 0.774, 1: 0.226}
Test set {0: 0.733, 1: 0.267}
```

So, the btsData is the standardized data that we use next to the classes data of 0 and 1. As that is our target. We use here the 80 20 split, and 70 30 splits as mentioned above as it's good practice. test size 0.2 means is that we 20% of the data for testing in the 80% for training. Next to that 30 percent for testing and 70% for training.

Task 4: Classification algorithms

4.1 Manually implement a KNN classifier. You are not allowed to use any predefined functions or libraries, except for auxiliary functionalities, such as the square root function of the Math library.

K-Nearest Neighbors (KNN) is a simple machine learning algorithm used for classification and regression. It works by storing training data and comparing new data points to their nearest neighbors. When a new point is introduced, KNN calculates the distance between it and the training points, then selects the K closest neighbors. For classification, it assigns the most common label among those neighbors. For regression, it averages the neighbors' values to make a prediction. The value of K determines how many neighbors are considered, which can affect the model's accuracy.

```
1 #Task 4.1: Manually implement KNN classifier - 80/20 set & 70/30 set
2
3 import numpy as np
4 from collections import Counter
5 import math
6
7 # Function to calculate Euclidean distance between two points
8 def euclidean_distance(point1, point2):
9     distance = 0.0
10    for i in range(len(point1)):
11        distance += (point1[i] - point2[i]) ** 2
12    return math.sqrt(distance)
13
14 # KNN classifier
15 class KNN:
16     def __init__(self, k=3):
17         self.k = k
18
19     # Fit function to store the training data
20     def fit(self, X_train, y_train):
21         self.X_train = np.array(X_train)
22         self.y_train = np.array(y_train)
23
24     # Function to predict the class of a single test point
25     def predict(self, X_test_point):
26         distances = []
27         # Calculate the distance from the test point to all training points
28         for i in range(len(self.X_train)):
29             dist = euclidean_distance(self.X_train[i], X_test_point)
30             distances.append((dist, self.y_train[i]))
31
32         # Sort by distance and get the top k nearest neighbors
33         distances.sort(key=lambda x: x[0])
34         neighbors = distances[:self.k]
35
36         # Get the most common class among the nearest neighbors
37         nearest_classes = [neighbor[1] for neighbor in neighbors]
38         prediction = Counter(nearest_classes).most_common(1)[0][0]
39
40         return prediction
41
42     # Function to predict classes for the entire test set
43     def predict_all(self, X_test):
44         predictions = []
45         for point in X_test:
46             predictions.append(self.predict(point))
47         return predictions
```

```

49 # Select a few cases from the test set
50 randomSamplesKNN = np.random.choice(X_test_80.index, 50, replace=False)
51 X_test_samplesKNN = X_test_80.loc[randomSamplesKNN]
52 y_test_samplesKNN = y_test_80.loc[randomSamplesKNN]
53
54 # Initialize and fit KNN model
55 knn = KNN(k=2)
56 knn.fit(X_train_80.values, y_train_80.values)
57
58 # Predict for the selected test samples
59 knnPredictions = knn.predict_all(X_test_samplesKNN.values)
60
61 # Compare predicted Labels with actual Labels
62 print("KNN Classifier 80/20:")
63 for i in range(len(X_test_samplesKNN)):
64     print(f"Test sample {i+1}: Predicted = {knnPredictions[i]}, Actual = {y_test_samplesKNN.iloc[i]}")
65
66 # Select a few cases from the test set
67 randomSamplesKNN2 = np.random.choice(X_test_70.index, 50, replace=False)
68 X_test_samplesKNN2 = X_test_70.loc[randomSamplesKNN2]
69 y_test_samplesKNN2 = y_test_70.loc[randomSamplesKNN2]
70
71 # Initialize and fit KNN model
72 knn = KNN(k=2)
73 knn.fit(X_train_70.values, y_train_70.values)
74
75 # Predict for the selected test samples
76 knnPredictions2 = knn.predict_all(X_test_samplesKNN2.values)
77
78 # Compare predicted Labels with actual Labels
79 print("\nKNN Classifier 70/30:")
80 for i in range(len(X_test_samplesKNN2)):
81     print(f"Test sample {i+1}: Predicted = {knnPredictions2[i]}, Actual = {y_test_samplesKNN2.iloc[i]}")
82
83
KNN Classifier 80/20:
Test sample 1: Predicted = 0, Actual = 0
Test sample 2: Predicted = 1, Actual = 0
Test sample 3: Predicted = 1, Actual = 0
Test sample 4: Predicted = 0, Actual = 0
Test sample 5: Predicted = 1, Actual = 0
Test sample 6: Predicted = 1, Actual = 0
Test sample 7: Predicted = 1, Actual = 0
Test sample 8: Predicted = 0, Actual = 0
Test sample 9: Predicted = 0, Actual = 0
Test sample 10: Predicted = 0, Actual = 0
Test sample 11: Predicted = 0, Actual = 0
Test sample 12: Predicted = 1, Actual = 0
Test sample 13: Predicted = 1, Actual = 0
Test sample 14: Predicted = 1, Actual = 0
Test sample 15: Predicted = 0, Actual = 0
Test sample 16: Predicted = 0, Actual = 0
Test sample 17: Predicted = 0, Actual = 0
Test sample 18: Predicted = 0, Actual = 0
Test sample 19: Predicted = 1, Actual = 0
Test sample 20: Predicted = 1, Actual = 0
Test sample 21: Predicted = 0, Actual = 1
Test sample 22: Predicted = 0, Actual = 0
Test sample 23: Predicted = 0, Actual = 0
Test sample 24: Predicted = 1, Actual = 0
Test sample 25: Predicted = 0, Actual = 0
Test sample 26: Predicted = 0, Actual = 0
Test sample 27: Predicted = 0, Actual = 0
Test sample 28: Predicted = 0, Actual = 0
Test sample 29: Predicted = 1, Actual = 1
Test sample 30: Predicted = 0, Actual = 1
Test sample 31: Predicted = 0, Actual = 0
Test sample 32: Predicted = 0, Actual = 0
Test sample 33: Predicted = 0, Actual = 0
Test sample 34: Predicted = 0, Actual = 0
Test sample 35: Predicted = 0, Actual = 0
Test sample 36: Predicted = 0, Actual = 0
Test sample 37: Predicted = 0, Actual = 1
Test sample 38: Predicted = 0, Actual = 0
Test sample 39: Predicted = 1, Actual = 0
Test sample 40: Predicted = 0, Actual = 1
Test sample 41: Predicted = 0, Actual = 0
Test sample 42: Predicted = 0, Actual = 0
Test sample 43: Predicted = 1, Actual = 1
Test sample 44: Predicted = 0, Actual = 0
Test sample 45: Predicted = 0, Actual = 0
Test sample 46: Predicted = 0, Actual = 0
Test sample 47: Predicted = 0, Actual = 1
Test sample 48: Predicted = 1, Actual = 0
Test sample 49: Predicted = 1, Actual = 0
Test sample 50: Predicted = 1, Actual = 0

KNN Classifier 70/30:
Test sample 1: Predicted = 0, Actual = 0

```

In this code, we manually implement a K-Nearest Neighbors (KNN) classifier to classify test samples using the 80/20 and 70/30 training-test splits. The `euclidean_distance` function computes the distance between a test point and each training point. The KNN class stores the training data and predicts the class of test samples by finding the k-nearest neighbors

and choosing the most common class among them. We first fit the KNN model with $k=2$ on the 80/20 split and then predict and compare 50 randomly selected test samples with their actual labels. The same process is repeated for the 70/30 split to evaluate model performance on different training-test splits.

4.2 Use Sklearn's Naive Bayes Classifier. Predict a few new cases and compare the predicted labels with the actual labels.

Naive Bayes is a simple and fast algorithm used for classification. It works by calculating the probability of each class for a given data point and then predicting the class with the highest probability. It assumes that all features (or inputs) are independent of each other, which makes the calculations easier. It's often used for tasks like spam filtering. Using Sklearn, you can quickly train a Naive Bayes model on your data, make predictions for new cases, and compare the predicted results with the actual labels to see how well the model performs.

```
1 #Task 4.2: Naive Bayes Classifier - 80/20 set & 70/30 set
2
3 from sklearn.naive_bayes import GaussianNB
4
5 # Initialize the Naive Bayes classification and train it on the 80/20 set
6 gnbClass80 = GaussianNB()
7 gnbClass80.fit(X_train_80, y_train_80)
8
9 # Initialize the Naive Bayes classification and train it on the 70/30 set
10 gnbClass70 = GaussianNB()
11 gnbClass70.fit(X_train_70, y_train_70)
12
13 # Select a few cases from the test set randomly
14 randomSamplesNBC = np.random.choice(X_test_80.index, 50, replace=False)
15 X_test_samplesNBC = X_test_80.loc[randomSamplesNBC]
16 y_test_samplesNBC = y_test_80.loc[randomSamplesNBC]
17
18 randomSamplesNBC2 = np.random.choice(X_test_70.index, 50, replace=False)
19 X_test_samplesNBC2 = X_test_70.loc[randomSamplesNBC2]
20 y_test_samplesNBC2 = y_test_70.loc[randomSamplesNBC2]
21
22
23 # Predict on the selected test samples
24 gnbPredictions = gnbClass80.predict(X_test_samplesNBC)
25 gnbPredictions2 = gnbClass70.predict(X_test_samplesNBC2)
26
27
28 # Compare predictions with actual labels
29 print("Naive Bayes Classifier Predictions 80/20:")
30 for i in range(50):
31     print(f"Test sample {i+1}: Predicted = {gnbPredictions[i]}, Actual = {y_test_samplesNBC.iloc[i]}")
32
33 # Compare predictions with actual labels
34 print("\nNaive Bayes Classifier Predictions 70/30:")
35 for i in range(50):
36     print(f"Test sample {i+1}: Predicted = {gnbPredictions2[i]}, Actual = {y_test_samplesNBC2.iloc[i]}")
37
```

Naive Bayes Classifier Predictions 80/20:
Test sample 1: Predicted = 1, Actual = 1
Test sample 2: Predicted = 0, Actual = 0
Test sample 3: Predicted = 0, Actual = 0

In the above code, a Naive Bayes Classifier is implemented using the GaussianNB model from the sklearn library. The model is trained separately on both the 80/20 and 70/30 training test splits. The training data for the 80/20 split is stored in X_train_80 (features) and y_train_80 (target labels), while for the 70/30 split, it is stored in X_train_70 and y_train_70. The gnbClass80 model is trained using the 80/20 data, and gnbClass70 is trained using the 70/30 data.

Next 50 random samples are selected from each of the test sets. For the 80/20 split, the indices of these samples are stored in randomSamplesNBC, and the corresponding test features and labels are placed in X_test_samplesNBC and Y_test_samplesNBC. Similarly, for the 70/30 split, the samples are selected and stored in randomSamplesNBC2, with the features and labels in X_test_samplesNBC2 and y_test_samplesNBC2.

After selecting the test samples, predictions are made using the models. For the 80/20 split, predictions are stored in gnbPredictions, while for the 70/30 split, predictions are stored in gnbPredictions2. Finally, the code compares these predictions with the actual labels, and prints the results for all 50 test samples, showing both predicted and actual class values for the two different train-test splits.

4.3 Use Sklearn's Support Vector Classifier. Predict a few new cases and compare the predicted labels with the actual labels.

Support Vector Classifier (SVC) is a powerful algorithm used for classification tasks. It works by finding a boundary that best separates the classes in your data. The goal is to maximize the margin between the data points from different classes. Using Sklearn, you can easily train an SVC model on your data, make predictions for new cases, and compare the predicted labels with the actual labels to evaluate how well the model is classifying the data.

```
1 #Task 4.3: Support Vector Classifier - 80/20 set & 70/30 set
2
3 from sklearn.svm import SVC
4 import numpy as np
5
6 # Initialize the Support Vector Classifier with class weights to handle imbalanced classes
7 svcData = SVC(class_weight='balanced')
8 svcData2 = SVC(class_weight='balanced')
9
10 # Train the model on the original 80-20 train set (no scaling)
11 svcData.fit(X_train_80, y_train_80)
12
13 # Train the model on the original 70-30 train set (no scaling)
14 svcData2.fit(X_train_70, y_train_70)
15
16 # Randomly select 5 samples from the test set
17 randomSamplesSVC = np.random.choice(X_test_80.index, 50, replace=False)
18 randomSamplesSVC2 = np.random.choice(X_test_70.index, 50, replace=False)
19
20 # Select the corresponding samples from the original test set
21 X_test_samplesSVC = X_test_80.loc[randomSamplesSVC]
22 y_test_samplesSVC = y_test_80.loc[randomSamplesSVC]
23
24 X_test_samplesSVC2 = X_test_70.loc[randomSamplesSVC2]
25 y_test_samplesSVC2 = y_test_70.loc[randomSamplesSVC2]
26
27 # Make predictions on the selected test samples
28 svcPredictions = svcData.predict(X_test_samplesSVC)
29 svcPredictions2 = svcData2.predict(X_test_samplesSVC2)
30
31 # Compare predictions with actual labels
32 print("SVC Classifier Predictions:")
33 for i in range(len(randomSamplesSVC)):
34     print(f"Test sample {i+1}: Predicted = {svcPredictions[i]}, Actual = {y_test_samplesSVC.iloc[i]}")
35
36 print("\nSVC Classifier Predictions:")
37 for i in range(len(randomSamplesSVC2)):
38     print(f"Test sample {i+1}: Predicted = {svcPredictions2[i]}, Actual = {y_test_samplesSVC2.iloc[i]}")
39
```

```
SVC Classifier Predictions:
Test sample 1: Predicted = 1, Actual = 0
Test sample 2: Predicted = 1, Actual = 1
Test sample 3: Predicted = 0, Actual = 0
Test sample 4: Predicted = 0, Actual = 0
Test sample 5: Predicted = 0, Actual = 0
```

In this code a Support Vector Classifier (SVC) is trained using the 80/20 and 70/30 train-test splits. The models `svcData` and `svcData2` are initialized with balanced class weights to handle imbalanced data. `svcData` is trained on the 80/20 set, and `svcData2` on the 70/30 set.

Next 50 random test samples are selected from both splits and stored in `X_test_samplesSVC` and `X_test_samplesSVC2`. The models make predictions on these samples, which are stored in `svcPredictions` and `svcPredictions2`. Finally, the predicted and actual labels for the test samples are compared and printed for both splits.

4.4 *Use Sklearn's Multilayer Perceptron (Neural Network) Classifier. Predict a few new cases and compare the predicted labels with the actual labels.*

The Multilayer Perceptron (MLP) is a type of neural network used for classification tasks. It consists of layers of neurons, where each neuron processes inputs to produce an output. Using Sklearn's `MLPClassifier`, you can train the model on your data, make predictions on new cases, and compare these predictions with the actual labels. This helps to assess how accurately the neural network is classifying the data.

```

1 #Task 4.4: Multilayer Perceptron (Neural Network) Classifier - 80/20 & 70/30 set
2
3 from sklearn.neural_network import MLPClassifier
4 import numpy as np
5
6 # Initialize the Multilayer Perceptron model
7 mlpData = MLPClassifier(max_iter=1000, random_state=42)
8 mlpData2 = MLPClassifier(max_iter=1000, random_state=42)
9
10 # Train the model on the original (non-scaled) 80-20 train set
11 mlpData.fit(X_train_80, y_train_80)
12
13 # Train the model on the original (non-scaled) 80-20 train set
14 mlpData2.fit(X_train_70, y_train_70)
15
16 # Randomly select 5 indices from the test set
17 randomSamplesMLP = np.random.choice(X_test_80.index, 50, replace=False)
18 randomSamplesMLP2 = np.random.choice(X_test_70.index, 50, replace=False)
19
20 # Select the corresponding non-scaled samples from X_test_20
21 X_test_samplesMLP = X_test_80.loc[randomSamplesMLP]
22 y_test_samplesMLP = y_test_80.loc[randomSamplesMLP]
23
24 # Select the corresponding non-scaled samples from X_test_30
25 X_test_samplesMLP2 = X_test_70.loc[randomSamplesMLP2]
26 y_test_samplesMLP2 = y_test_70.loc[randomSamplesMLP2]
27
28 # Make predictions on the selected test samples
29 mlpPredictions = mlpData.predict(X_test_samplesMLP)
30 mlpPredictions2 = mlpData2.predict(X_test_samplesMLP2)
31
32 # Compare predictions with actual Labels
33 print("MLP Classifier Predictions:")
34 for i in range(len(randomSamplesMLP)):
35     print(f"Test sample {i+1}: Predicted = {mlpPredictions[i]}, Actual = {y_test_samplesMLP.iloc[i]}")
36
37 # Compare predictions with actual Labels
38 print("\nMLP Classifier Predictions:")
39 for i in range(len(randomSamplesMLP2)):
40     print(f"Test sample {i+1}: Predicted = {mlpPredictions2[i]}, Actual = {y_test_samplesMLP2.iloc[i]}")

```

```

MLP Classifier Predictions:
Test sample 1: Predicted = 1, Actual = 0
Test sample 2: Predicted = 0, Actual = 1
Test sample 3: Predicted = 0, Actual = 0
Test sample 4: Predicted = 1, Actual = 0
Test sample 5: Predicted = 0, Actual = 0
Test sample 6: Predicted = 0, Actual = 0
Test sample 7: Predicted = 0, Actual = 0

```

In this task, a Multilayer Perceptron (MLP) classifier is created using the `MLPClassifier` from `sklearn`, which functions as a neural network model. Two models are initialized, `mlpData` and `mlpData2`, both with a limit of 1000 iterations and a fixed random state to ensure repeatability. The first model is trained on the 80/20 training split, while the second model is trained using the 70/30 split.

Once the models are trained, the code randomly selects 50 test samples from both the 80/20 and 70/30 splits, storing them in `X_test_samplesMLP` and `X_test_samplesMLP2`. The true labels for these samples are kept in `y_test_samplesMLP` and `y_test_samplesMLP2`.

Predictions are then made on the selected test data by both models. The results of these predictions, stored in `mlpPredictions` and `mlpPredictions2`, are then compared to the actual test labels. The script proceeds to display the predicted vs. actual values for each sample, showing how the MLP performed in both splits.

Task 5: Evaluation of classification methods

5.1 Manually implement a confusion matrix to evaluate the results of the classification models. You are not allowed to use any predefined functions or libraries.

A confusion matrix is a simple way to evaluate how well a classification model is working. It shows how many times the model correctly predicted a label with the values: Actual Positive and Actual Negative and how many times it made errors using Predicted Positive and Predicted Negative. By manually implementing a confusion matrix, you calculate these values yourself without using built-in functions, giving you a clearer understanding of the model's performance. It shows the performance of the 80/20 split and 70/30 split for every classification.

```
1 #Task 5.1: Manually implement confusion matrix
2
3 # Reusable confusion matrix functions
4 def confusion_matrix_manual(y_true, y_pred, labels):
5     # Initialize the confusion matrix with zeros
6     num_labels = len(labels)
7     matrix = [[0 for _ in range(num_labels)] for _ in range(num_labels)]
8
9     # Create a dictionary to map class labels to indices
10    label_to_index = {label: i for i, label in enumerate(labels)}
11
12    # Iterate through true and predicted labels
13    for true_label, pred_label in zip(y_true, y_pred):
14        i = label_to_index[true_label] # Index of true label
15        j = label_to_index[pred_label] # Index of predicted label
16        matrix[i][j] += 1 # Increment the corresponding cell
17
18    return matrix
19
20 def print_confusion_matrix(matrix, labels):
21    print("Confusion Matrix:")
22    print(f"{'':<20}{'Predicted Negative':<20}{'Predicted Positive':<20}")
23    for i, row in enumerate(matrix):
24        actual_label = 'Actual Negative' if i == 0 else 'Actual Positive'
25        print(f"{actual_label:<20}" + ' '.join([f'{item:<20}' for item in row]))
26
```

In this task we're building a confusion matrix from scratch, a crucial tool for evaluating the performance of a classification model. The function `confusion_matrix_manual` starts by initializing an empty matrix with zeros. This matrix will hold counts of how many instances were correctly or incorrectly classified. For instance the diagonal of the matrix shows correct predictions, while off-diagonal entries reveal misclassifications.

A dictionary is used to map each class label to an index, allowing us to populate the matrix accurately. The function then loops through the actual (`y_true`) and predicted (`y_pred`) labels. For each comparison it identifies the corresponding matrix cell based on the actual and predicted class and increments the count there.

After building the matrix, `print_confusion_matrix` presents it in a readable way, labeling rows as "Actual Negative" and "Actual Positive," while columns are marked as "Predicted Negative" and "Predicted Positive." This format makes it easy to interpret where the classifier performed well and where it struggled. It's a creative approach to get an in-depth

view of how well your model is handling classification tasks, going beyond just accuracy scores!

```
1 # Example usage with KNN classifier predictions
2 # 80/20 Split
3 conf_matrix_knn_80_20 = confusion_matrix_manual(y_test_samplesKNN, knnPredictions, labels=[0, 1])
4 print("KNN Classifier Confusion Matrix (80/20 Split):")
5 print_confusion_matrix(conf_matrix_knn_80_20, labels=[0, 1])
6
7 # 70/30 Split
8 conf_matrix_knn_70_30 = confusion_matrix_manual(y_test_samplesKNN2, knnPredictions2, labels=[0, 1])
9 print("\nKNN Classifier Confusion Matrix (70/30 Split):")
10 print_confusion_matrix(conf_matrix_knn_70_30, labels=[0, 1])
11
12 # 80/20 Split: Confusion Matrix for Naive Bayes
13 conf_matrix_gnb_80_20 = confusion_matrix_manual(y_test_samplesNBC, gnbPredictions, labels=[0, 1])
14 print("\nNaive Bayes Classifier Confusion Matrix (80/20 Split):")
15 print_confusion_matrix(conf_matrix_gnb_80_20, labels=[0, 1])
16
17 # 70/30 Split: Confusion Matrix for Naive Bayes
18 conf_matrix_gnb_70_30 = confusion_matrix_manual(y_test_samplesNBC2, gnbPredictions2, labels=[0, 1])
19 print("\nNaive Bayes Classifier Confusion Matrix (70/30 Split):")
20 print_confusion_matrix(conf_matrix_gnb_70_30, labels=[0, 1])
21
22 conf_matrix_svc_80_20 = confusion_matrix_manual(y_test_samplesSVC, svcPredictions, labels=[0, 1])
23 print("\nSupport Vector Classifier Confusion Matrix (80/20 Split):")
24 print_confusion_matrix(conf_matrix_svc_80_20, labels=[0, 1])
25
26 # 70/30 Split: Confusion Matrix for SVC
27 conf_matrix_svc_70_30 = confusion_matrix_manual(y_test_samplesSVC2, svcPredictions2, labels=[0, 1])
28 print("\nSupport Vector Classifier Confusion Matrix (70/30 Split):")
29 print_confusion_matrix(conf_matrix_svc_70_30, labels=[0, 1])
30
31 # 80/20 Split: Confusion Matrix for MLP
32 conf_matrix_mlp_80_20 = confusion_matrix_manual(y_test_samplesMLP, mlpPredictions, labels=[0, 1])
33 print("\nMultilayer Perceptron Confusion Matrix (80/20 Split):")
34 print_confusion_matrix(conf_matrix_mlp_80_20, labels=[0, 1])
35
36 # 70/30 Split: Confusion Matrix for MLP
37 conf_matrix_mlp_70_30 = confusion_matrix_manual(y_test_samplesMLP2, mlpPredictions2, labels=[0, 1])
38 print("\nMultilayer Perceptron Confusion Matrix (70/30 Split):")
39 print_confusion_matrix(conf_matrix_mlp_70_30, labels=[0, 1])
```

KNN Classifier Confusion Matrix (80/20 Split):

Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	28	15
Actual Positive	5	2

KNN Classifier Confusion Matrix (70/30 Split):

Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	29	6
Actual Positive	11	4

Naive Bayes Classifier Confusion Matrix (80/20 Split):

5.2 Use Sklearn's classification report function to evaluate the results of the classification models.

Sklearn's classification report function is a built-in tool that helps evaluate how well a classification model is performing. It summarizes key metrics like precision, recall, and F1-score for each class. These metrics help you understand how often the model is making correct predictions, how sensitive it is to positive cases, and its overall accuracy. Using this function gives you a quick and detailed evaluation of the model's strengths and weaknesses. When filling in the built-in tool, you provide the sample of the test split, the predictions of the model and when a division of zero occurs the value will return 0. For example, this can happen when there are no positive predictions made or no actual positive labels.

```
1 #Task5.2: Classification report - 80/20 & 70/30 set
2
3 from sklearn.metrics import classification_report
4
5 # SVC Classification Report
6 print("SVC Classification Report 80/20:")
7 print(classification_report(y_test_samplesSVC, svcPredictions, zero_division=0))
8 print("SVC Classification Report 70/30:")
9 print(classification_report(y_test_samplesSVC2, svcPredictions2, zero_division=0))
10
11 # Naive Bayes Classification Report
12 print("\nNaive Bayes Classification Report 80/20:")
13 print(classification_report(y_test_samplesNBC, gnbPredictions, zero_division=0))
14
15 print("\nNaive Bayes Classification Report 70/30:")
16 print(classification_report(y_test_samplesNBC2, gnbPredictions2, zero_division=0))
17
18 # MLP Classification Report
19 print("\nMLP Classification Report 80/20:")
20 print(classification_report(y_test_samplesMLP, mlpPredictions, zero_division=0))
21
22 print("\nMLP Classification Report 70/30:")
23 print(classification_report(y_test_samplesMLP2, mlpPredictions2, zero_division=0))
24
25 # KNN Classification Report
26 print("\nKNN Classification Report 80/20:")
27 print(classification_report(y_test_samplesKNN, knnPredictions, zero_division=0))
28
29 print("\nKNN Classification Report 70/30:")
30 print(classification_report(y_test_samplesKNN2, knnPredictions2, zero_division=0))
31
```

```
SVC Classification Report 80/20:
      precision    recall  f1-score   support

     0       0.88      0.61      0.72        38
     1       0.38      0.75      0.50        12

 accuracy          0.63
 macro avg          0.63
 weighted avg       0.76
```

```
SVC Classification Report 70/30:
      precision    recall  f1-score   support

     0       0.83      0.69      0.75        35
     1       0.48      0.67      0.56        15

 accuracy          0.65
 macro avg          0.65
 weighted avg       0.72
```

```
Naive Bayes Classification Report 80/20:
```


5.3 Use Sklearn's fbeta score function to evaluate the results of the classification models.

Sklearn's fbeta score function is used to evaluate a classification model by balancing precision and recall with a customizable emphasis. The F-beta score helps measure the model's performance by assigning different weights to recall and precision. If you want to focus more on recall, you can increase the value of beta, and if you want to prioritize precision, you can lower it. This function is useful when you want to fine-tune the balance between detecting positive cases and avoiding false alarms.

```
1 #Task 5.3: Fbeta score - 80/20 & 70/30 set
2
3 from sklearn.metrics import fbeta_score
4
5 # SVC F-beta scores 80/20
6 svc_fbeta1 = fbeta_score(y_test_samplesSVC, svcPredictions, beta=1, zero_division=0)
7 print(f"SVC F1-score 80/20: {svc_fbeta1}")
8 svc_fbeta2 = fbeta_score(y_test_samplesSVC, svcPredictions, beta=2, zero_division=0)
9 print(f"SVC F-beta score 80/20: {svc_fbeta2}")
10
11 # SVC F-beta scores 70/30
12 svc_fbeta1 = fbeta_score(y_test_samplesSVC2, svcPredictions2, beta=1, zero_division=0)
13 print(f"SVC F1-score 70/30: {svc_fbeta1}")
14 svc_fbeta2 = fbeta_score(y_test_samplesSVC2, svcPredictions2, beta=2, zero_division=0)
15 print(f"SVC F-beta score 70/30: {svc_fbeta2}")
16
17 # Naive Bayes F-beta scores 80/20
18 nb_fbeta1 = fbeta_score(y_test_samplesNBC, gnbPredictions, beta=1, zero_division=0)
19 print(f"Naive Bayes F1-score 80/20: {nb_fbeta1}")
20 nb_fbeta2 = fbeta_score(y_test_samplesNBC, gnbPredictions, beta=2, zero_division=0)
21 print(f"Naive Bayes F-beta score 80/20: {nb_fbeta2}")
22
23 # Naive Bayes F-beta scores 70/30
24 nb_fbeta1 = fbeta_score(y_test_samplesNBC2, gnbPredictions2, beta=1, zero_division=0)
25 print(f"Naive Bayes F1-score 70/30: {nb_fbeta1}")
26 nb_fbeta2 = fbeta_score(y_test_samplesNBC2, gnbPredictions2, beta=2, zero_division=0)
27 print(f"Naive Bayes F-beta score 70/30: {nb_fbeta2}")
28
29 # MLP F-beta scores 80/20
30 mlp_fbeta1 = fbeta_score(y_test_samplesMLP, mlpPredictions, beta=1, zero_division=0)
31 print(f"MLP F1-score 80/20: {mlp_fbeta1}")
32 mlp_fbeta2 = fbeta_score(y_test_samplesMLP, mlpPredictions, beta=2, zero_division=0)
33 print(f"MLP F-beta score 80/20: {mlp_fbeta2}")
34
35 # MLP F-beta scores 70/30
36 mlp_fbeta1 = fbeta_score(y_test_samplesMLP2, mlpPredictions2, beta=1, zero_division=0)
37 print(f"MLP F1-score 70/30: {mlp_fbeta1}")
38 mlp_fbeta2 = fbeta_score(y_test_samplesMLP2, mlpPredictions2, beta=2, zero_division=0)
39 print(f"MLP F-beta score 70/30: {mlp_fbeta2}")
40
41 # KNN F-beta scores 80/20
42 knn_fbeta1 = fbeta_score(y_test_samplesKNN, knnPredictions, beta=1, zero_division=0)
43 print(f"KNN F1-score 80/20: {knn_fbeta1}")
44 knn_fbeta2 = fbeta_score(y_test_samplesKNN, knnPredictions, beta=2, zero_division=0)
45 print(f"KNN F-beta score 80/20: {knn_fbeta2}")
46
47 # KNN F-beta scores 70/30
48 knn_fbeta1 = fbeta_score(y_test_samplesKNN2, knnPredictions2, beta=1, zero_division=0)
49 print(f"KNN F1-score 70/30: {knn_fbeta1}")
50 knn_fbeta2 = fbeta_score(y_test_samplesKNN2, knnPredictions2, beta=2, zero_division=0)
51 print(f"KNN F-beta score 70/30: {knn_fbeta2}")
52
```

```
SVC F1-score 80/20: 0.5
SVC F-beta score 80/20: 0.625
SVC F1-score 70/30: 0.5555555555555556
SVC F-beta score 70/30: 0.617283950617284

Naive Bayes F1-score 80/20: 0.125
Naive Bayes F-beta score 80/20: 0.09615384615384615
Naive Bayes F1-score 70/30: 0.2
Naive Bayes F-beta score 70/30: 0.15384615384615383

MLP F1-score 80/20: 0.3157894736842105
MLP F-beta score 80/20: 0.25862068965517243
MLP F1-score 70/30: 0.1818181818181818
MLP F-beta score 70/30: 0.1923076923076923

KNN F1-score 80/20: 0.16666666666666666
KNN F-beta score 80/20: 0.2222222222222222
KNN F1-score 70/30: 0.32
KNN F-beta score 70/30: 0.2857142857142857
```

Task 6: Cross-validation

6.1 Apply Sklearn's K-Fold cross-validation on both splits of one of the classification algorithms to optimize at least one parameter of the algorithm you have chosen. Use scores from task 5 when evaluating each fold. Afterwards, refit the classifier, this time with the best parameter setting. Render plots showing the evaluation scores for each fold. Also render plot(s) showing the difference between not using cross-validation and using cross-validation for this algorithm with both splits.

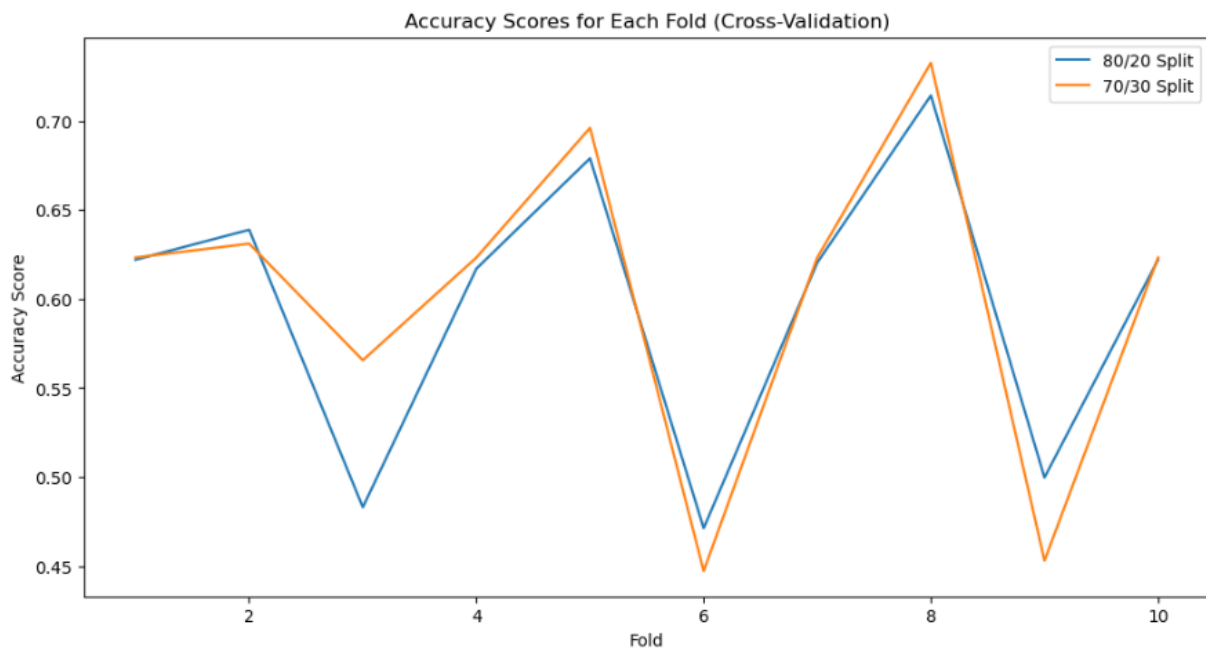
K-Fold cross-validation is a method to assess how well a classification model performs by dividing the dataset into 'k' parts, or folds. The model is trained on 'k-1' folds and tested on the remaining fold, allowing every data point to be used for both training and testing. This process helps to provide a more accurate evaluation of the model's performance.

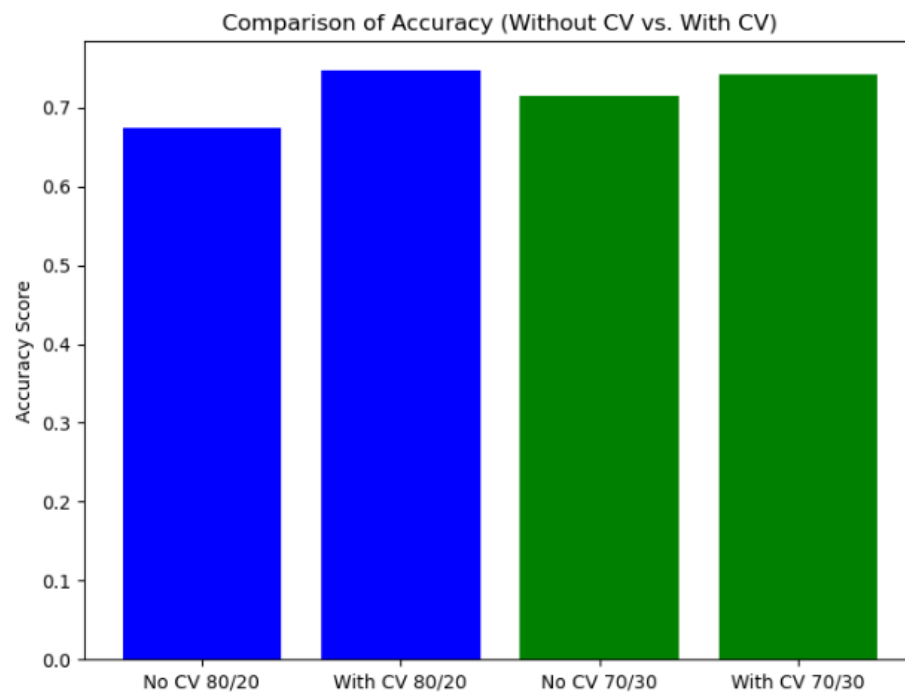
```
1 from sklearn.model_selection import KFold, GridSearchCV
2 from sklearn.svm import SVC
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # Initialize the Support Vector Classifier with class weights to handle imbalanced classes
7 svc = SVC(class_weight='balanced')
8
9 # Define parameter grid for optimization (e.g., regularization parameter C and kernel type)
10 param_grid = {'C': [0.1, 1, 10, 100], 'kernel': ['linear', 'rbf', 'poly']}
11
12 # Set up KFold cross-validation (5 folds)
13 kf = KFold(n_splits=10, shuffle=True, random_state=42)
14
15 # Set up GridSearchCV to optimize hyperparameters using cross-validation
16 grid_search_80_20 = GridSearchCV(estimator=svc, param_grid=param_grid, cv=kf, scoring='accuracy')
17 grid_search_70_30 = GridSearchCV(estimator=svc, param_grid=param_grid, cv=kf, scoring='accuracy')
18
19 # Fit the model using cross-validation and the best parameter combination for both 80/20 and 70/30 splits
20 grid_search_80_20.fit(X_train_80, y_train_80)
21 grid_search_70_30.fit(X_train_70, y_train_70)
22
23 # Store best parameters and corresponding scores
24 best_params_80_20 = grid_search_80_20.best_params_
25 best_params_70_30 = grid_search_70_30.best_params_
26
27 # Refit the model using the best parameters found from cross-validation
28 best_svc_80_20 = grid_search_80_20.best_estimator_
29 best_svc_70_30 = grid_search_70_30.best_estimator_
30
31 # Get cross-validation results for each fold
32 cv_results_80_20 = grid_search_80_20.cv_results_['mean_test_score']
33 cv_results_70_30 = grid_search_70_30.cv_results_['mean_test_score']
34
35 # Plotting cross-validation results for both splits
36 plt.figure(figsize=(12, 6))
37 plt.plot(range(1, 11), cv_results_80_20[:10], label='80/20 Split')
38 plt.plot(range(1, 11), cv_results_70_30[:10], label='70/30 Split')
39 plt.xlabel('Fold')
40 plt.ylabel('Accuracy Score')
41 plt.title('Accuracy Scores for Each Fold (Cross-Validation)')
42 plt.legend()
43 plt.show()
44
45 # Now, Let's compare the scores before and after cross-validation
46 svc_no_cv_80_20 = SVC(class_weight='balanced')
47 svc_no_cv_70_30 = SVC(class_weight='balanced')
48
49 # Train the model without cross-validation
50 svc_no_cv_80_20.fit(X_train_80, y_train_80)
51 svc_no_cv_70_30.fit(X_train_70, y_train_70)
```

```

52
53 # Evaluate the models without cross-validation on the test sets
54 score_no_cv_80_20 = svc_no_cv_80_20.score(X_test_80, y_test_80)
55 score_no_cv_70_30 = svc_no_cv_70_30.score(X_test_70, y_test_70)
56
57 # Evaluate the models after cross-validation on the test sets
58 score_cv_80_20 = best_svc_80_20.score(X_test_80, y_test_80)
59 score_cv_70_30 = best_svc_70_30.score(X_test_70, y_test_70)
60
61 # Plotting the comparison between no CV and with CV
62 scores_before_after_cv = {
63     'No CV 80/20': score_no_cv_80_20,
64     'With CV 80/20': score_cv_80_20,
65     'No CV 70/30': score_no_cv_70_30,
66     'With CV 70/30': score_cv_70_30,
67 }
68
69 plt.figure(figsize=(8, 6))
70 plt.bar(scores_before_after_cv.keys(), scores_before_after_cv.values(), color=['blue', 'blue', 'green', 'green'])
71 plt.ylabel('Accuracy Score')
72 plt.title('Comparison of Accuracy (Without CV vs. With CV)')
73 plt.show()
74
75 # Print the best parameters found for both splits
76 print("Best parameters for 80/20 split:", best_params_80_20)
77 print("Best parameters for 70/30 split:", best_params_70_30)
78

```





Best parameters for 80/20 split: {'C': 10, 'kernel': 'rbf'}
Best parameters for 70/30 split: {'C': 10, 'kernel': 'rbf'}
